

Searchable Secure Encrypted Block Storage Service

Segurança de Sistemas Computacionais

MADALENA FORTUNATO ALVES

MADALENA FORTUNATO ALVES - 75150

HENRIQUE MONTEIRO - 72928

MADALENA FORTUNATO ALVES

Relatório do Projeto – Searchable Secure Encrypted Block Storage Service

Introdução

Este relatório descreve o desenvolvimento de um sistema cliente-servidor para armazenamento de ficheiros em blocos cifrados, com suporte para pesquisa segura (Searchable Encryption). O projeto foi realizado no âmbito da unidade curricular de Segurança de Sistemas Computacionais, e implementa os requisitos definidos no Work Assignment 1: Searchable Secure Encrypted Block Storage Service.

O objetivo é construir uma aplicação de armazenamento remoto que assegure confidencialidade, integridade e autenticidade dos dados transferidos entre cliente e servidor. Os ficheiros são divididos em blocos, cifrados localmente e armazenados no servidor de forma opaca. O servidor não possui acesso ao conteúdo em claro, funcionando apenas como repositório persistente de blocos cifrados e metadados associados.

Além da proteção dos dados, o sistema implementa um mecanismo de pesquisa baseado em tokens criptográficos que permite procurar ficheiros por palavras-chave sem revelar essas keywords ao servidor.

Arquitetura do Sistema

A aplicação segue uma arquitetura cliente-servidor baseada em comunicação TCP. O cliente é responsável por cifrar e autenticar os dados, gerir as chaves criptográficas e manter um índice local com metadados. O servidor armazena blocos cifrados e metadados de forma persistente, sem nunca aceder ao conteúdo original.

A comunicação é feita através de mensagens JSON enviadas sobre sockets TCP. Cada mensagem inclui um comando e os respetivos parâmetros (por exemplo, PUT, GET, LIST, SEARCH). O servidor interpreta o comando e responde com o resultado em formato JSON.

A estrutura do projeto é composta por quatro diretórios principais:

- client/: código do cliente e ficheiros de configuração (cryptoconfig.txt, enc_key.txt, mac_key.txt);
- server/: código do servidor, chaves e armazenamento cifrado (blocks/ e metadata/);
- test/: scripts e ficheiros de teste (cltestcopy.py, Run_testes.py, commands.txt);
- documentation/: documentação técnica e relatório final.

O servidor mantém a persistência dos blocos e metadados entre execuções. Todos os dados são guardados em formato cifrado no disco, cumprindo o requisito de armazenamento seguro e persistente.

Implementação Criptográfica

A componente criptográfica utiliza o modo AES-GCM (Galois/Counter Mode) para garantir simultaneamente confidencialidade, integridade e autenticação das mensagens. Este modo de operação é adequado para sistemas cliente-servidor, dado que a verificação de integridade é automática através do authentication tag.

O ficheiro `cryptoconfig.txt` define os parâmetros do sistema: tamanho das chaves (256 bits), tamanho dos blocos, modo de nonce (aleatório ou determinístico) e ativação da deduplicação (DEDUP=ON/OFF).

Quando a deduplicação está ativa, o nonce é derivado de forma determinística a partir do hash SHA-256 do conteúdo do bloco, o que permite evitar a duplicação de blocos iguais. Caso contrário, o nonce é aleatório, garantindo segurança sem revelar padrões de repetição.

O cliente possui duas chaves principais:

- `enc_key.txt`: chave simétrica para cifragem de blocos com AES-GCM;
- `mac_key.txt`: chave para geração de tokens HMAC-SHA256 usados na pesquisa segura.

As funções criptográficas principais são:

- `encrypt_block()` e `decrypt_block()`: realizam a cifragem e decifragem com AES-GCM;
- `token_from_keyword()`: gera tokens determinísticos HMAC-SHA256 a partir de keywords;
- `name_from_ciphertext()`: obtém identificadores opacos para blocos com SHA-256;
- `encrypt_file_id()` e `decrypt_file_id()`: protegem os identificadores dos ficheiros armazenados.

Cliente

O cliente, implementado em Python, é responsável pelas operações de cifragem, armazenamento local e comunicação com o servidor. Na inicialização, o programa lê `cryptoconfig.txt` e valida os tamanhos das chaves. O índice local é mantido em `index/client_index.ser` e armazena metadados sobre os ficheiros e as respetivas keywords.

A operação PUT lê o ficheiro local, divide-o em blocos e cifra cada bloco com AES-GCM. O nome de cada bloco é obtido pelo hash SHA-256 do ciphertext. Os blocos são enviados ao servidor através do comando STORE_BLOCK. Em seguida, o cliente gera um identificador cifrado para o ficheiro e um token HMAC-SHA256 para cada keyword, enviados através do comando STORE_TOKEN. Este processo associa as keywords aos ficheiros de forma segura.

Na operação GET, o cliente pode recuperar um ficheiro completo pelo nome ou por uma keyword. O servidor responde com os blocos cifrados, e o cliente verifica a integridade através da tag GCM antes de decifrar. Se algum bloco tiver sido modificado, o processo é interrompido.

A função CHECKINTEGRITY executa uma verificação de integridade completa: descarrega novamente os blocos e tenta decifrá-los para validar as tags GCM. Além disso, confirma se os tokens registados no servidor correspondem aos ficheiros armazenados localmente.

As operações LIST e SEARCH consultam, respetivamente, os ficheiros no índice local e os ficheiros associados a uma keyword. No caso de SEARCH, o cliente envia apenas o token HMAC da palavra-chave, sem expor o termo original.

Servidor

O servidor, também em Python, é uma aplicação multithreaded que gere múltiplas conexões TCP em simultâneo. Cada cliente é tratado por uma thread independente, permitindo execução concorrente.

O armazenamento é dividido em duas pastas principais:

- server/storage/blocks: blocos cifrados;
- server/storage/metadata: ficheiros index.json e filemap.json, cifrados com AES-GCM usando a chave armazenada em server/keys/key.txt.

A função store_block() grava blocos no disco, assegurando persistência e deduplicação. Já store_token() mantém o índice de pesquisa e o mapeamento entre tokens, ficheiros e blocos.

As operações do servidor incluem:

STORE_BLOCK e STORE_TOKEN: armazenamento de blocos e metadados;

GET: devolve blocos cifrados de um ficheiro ou resultados de pesquisa;

SEARCH: devolve identificadores cifrados de ficheiros correspondentes a um token;

LIST_BLOCKS: lista os blocos armazenados (usado para teste e depuração).

Os acessos a ficheiros são controlados com locks para garantir consistência em ambiente multithread. A escrita é atômica e protegida, utilizando funções auxiliares como `_aesgcm_encrypt_json()` e `_atomic_write()`. Os nomes dos blocos e ficheiros são opacos e baseados em hashes, impedindo que a estrutura do diretório revele informação sobre o conteúdo.

Testes e Validação

A validação foi feita através de scripts automáticos disponíveis na pasta `test/`. O ficheiro `cltestcopy.py` fornece uma interface de linha de comandos para execução das operações PUT, GET, LIST, SEARCH e CHECKINTEGRITY.

Os scripts `Run_testes.py` executam sequências de testes definidos em `commands.txt`, incluindo upload e download de ficheiros, listagem, pesquisa e verificação de integridade.

Foram testadas situações de concorrência, com múltiplos clientes a realizar operações em simultâneo. A implementação multithread do servidor respondeu corretamente a todos os pedidos, sem conflitos de escrita. Também foi validada a deduplicação: quando `DEDUP=ON`, blocos idênticos resultam no mesmo ciphertext e não são duplicados no servidor.

Durante os testes, confirmou-se que os dados armazenados são totalmente opacos. Os blocos cifrados e metadados não contêm informação legível nem indicam o tipo de ficheiro. A função CHECKINTEGRITY deteta qualquer modificação, assegurando a integridade e autenticidade do armazenamento.

Garantias de Segurança e Conclusão

- A implementação cumpre os requisitos definidos no enunciado.
- A confidencialidade é garantida por AES-GCM, que cifra blocos e metadados.
- A integridade e autenticidade são asseguradas pelas tags GCM e pelos HMACs.
- A pesquisa segura é implementada com tokens determinísticos HMAC-SHA256, permitindo que o servidor processe consultas sem conhecer as keywords originais.
- A resistência a adversários honest-but-curious é obtida porque o servidor nunca tem acesso a informação em claro.
- A deduplicação, a persistência de dados e o suporte a múltiplas conexões foram implementados e testados com sucesso.

O projeto cumpre integralmente as operações obrigatórias (PUT, GET, LIST, SEARCH, CHECKINTEGRITY) e os objetivos de segurança. O sistema demonstra a

aplicação prática de mecanismos de criptografia autenticada, integridade de dados e pesquisa cifrada em ambiente distribuído.